Fundamentos de la Simulación de Monte Carlo

Alejandra Tabares Pozos

1. Introducción

La Simulación de Monte Carlo es una técnica numérica que utiliza el muestreo aleatorio repetido para obtener resultados numéricos. Es esencialmente un método para resolver problemas deterministas o probabilísticos mediante la simulación de variables aleatorias. En el contexto de la ingeniería industrial, financiera y comercial, esta herramienta nos permite modelar la incertidumbre inherente a los sistemas complejos, donde las soluciones analíticas cerradas pueden ser difíciles o imposibles de derivar.

El nombre "Monte Carlo"hace referencia al famoso casino en Mónaco, aludiendo al elemento de azar que es intrínseco a este método. Su desarrollo moderno se atribuye a científicos como Stanislaw Ulam y John von Neumann durante el desarrollo del Proyecto Manhattan en la década de 1940.

2. Fundamento Teórico

La base matemática de la simulación de Monte Carlo descansa sobre la **Ley de los Grandes Números**. En términos simples, este teorema establece que si repetimos un experimento aleatorio un número suficientemente grande de veces, el promedio de los resultados obtenidos convergerá al valor esperado (media teórica) de la distribución subyacente.

Formalmente, sea $X_1, X_2, \ldots, X_n$ una secuencia de variables aleatorias independientes e idénticamente distribuidas (i.i.d.) con valor esperado μ. La Ley Fuerte de los Grandes Números establece que el promedio muestral $\bar{X}_n$ converge casi seguramente a μ cuando $n \to \infty$:

$$P\left(\lim_{n\to\infty} \bar{X}_n = \mu\right) = 1 \tag{1}$$

Esto implica que podemos aproximar integrales complejas o valores esperados de funciones de variables aleatorias mediante el promedio de muestras generadas computacionalmente.

3. Ejemplo Introductorio: Estimación de π

Para ilustrar el mecanismo básico sin la complejidad de distribuciones de probabilidad avanzadas, consideremos un problema geométrico: la estimación del valor de π.

3.1. Planteamiento

Imaginemos un cuadrado de lado 2 centrado en el origen $(0, 0)$, con un área total de 4. Dentro de este cuadrado, inscribimos un círculo de radio $r = 1$. El área del círculo es $A_c = \pi r^2 = \pi$.

Si generamos puntos aleatorios uniformemente distribuidos dentro del cuadrado, la probabilidad P de que un punto caiga dentro del círculo es proporcional a la razón de las áreas:

$$P(\text{dentro}) = \frac{\text{Área del Círculo}}{\text{Área del Cuadrado}} = \frac{\pi(1)^2}{2 \times 2} = \frac{\pi}{4} \tag{2}$$

Por lo tanto, si lanzamos N puntos totales y contamos que M puntos caen dentro del círculo, podemos aproximar π como:

$$\pi \approx 4 \times \frac{M}{N} \tag{3}$$

3.2. Implementación en Python

A continuación, se presenta el código para realizar esta simulación.

```python
import random

def estimar_pi_monte_carlo(n_iteraciones):
    puntos_dentro = 0

    for _ in range(n_iteraciones):
        # Generamos coordenadas x e y entre -1 y 1
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)

        # Verificamos si el punto esta dentro del circulo unitario
        # x^2 + y^2 <= r^2 (donde r=1)
        if x**2 + y**2 <= 1:
            puntos_dentro += 1

    estimacion = 4 * (puntos_dentro / n_iteraciones)
    return estimacion

# Ejecucion con diferentes tamanos de muestra
n_vals = [1000, 10000, 1000000]
for n in n_vals:
    pi_aprox = estimar_pi_monte_carlo(n)
    print(f"Con n={n}, Pi se aproxima a: {pi_aprox:.5f}")
```

Listing 1: Estimación de Pi mediante Monte Carlo

Al aumentar N, la aproximación se vuelve más precisa, convergiendo al valor real de 3,14159...

4. Elementos Estructurales de una Simulación de Monte Carlo

Independientemente de si estamos modelando el precio de una opción financiera, el flujo de neutrones en un reactor o la línea de espera en una fábrica, toda simulación de Monte Carlo comparte una arquitectura común. Identificar estos componentes permite al ingeniero diseñar modelos modulares y verificables.

Podemos distinguir cinco elementos fundamentales en el ciclo de vida de una iteración de Monte Carlo:

4.1. Definición de las Distribuciones de Probabilidad de Entrada

El primer paso es identificar las variables estocásticas del sistema $(X_1, X_2, \ldots, X_k)$ y asignarles una distribución de probabilidad teórica o empírica que describa su comportamiento.

- *Ejemplo:* En el caso de negocio anterior, definimos la demanda como $N(\mu, \sigma)$ y el costo como $U(a, b)$.
- *Formalismo:* Debemos conocer la Función de Densidad de Probabilidad (PDF), $f(x)$, o la Función de Distribución Acumulada (CDF), $F(x)$, para cada variable.

4.2. Generación de Números Aleatorios (RNG)

Es el motor de la simulación. Se requiere una fuente de números aleatorios distribuidos uniformemente en el intervalo $(0, 1)$, denotados como $U \sim \text{Uniforme}(0, 1)$.

- Estos números sirven como base para generar valores de cualquier otra distribución mediante métodos como la *Transformada Inversa* o *Aceptación-Rechazo*.

4.3. Regla de Muestreo (Generación de Variadas)

Es el algoritmo que transforma el número aleatorio uniforme U en una variable aleatoria con la distribución deseada definida en el paso 1.

$$x_i = F^{-1}(u_i) \tag{4}$$

Donde F^{-1} es la inversa de la función de distribución acumulada. Computacionalmente, librerías como `numpy` automatizan este paso (ej. `np.random.normal`).

4.4. Modelo Computacional (Función de Transferencia)

Es la representación matemática determinista del sistema. Toma las variables aleatorias generadas (X) como entrada y produce una variable de respuesta (Y).

$$Y = h(X_1, X_2, \ldots, X_k) \tag{5}$$

Si el modelo es complejo, $h(\cdot)$ puede no ser una simple ecuación algebraica, sino una simulación de eventos discretos completa o un análisis de elementos finitos.

4.5. Agregación y Análisis de Error

Finalmente, se recolectan las salidas $Y_1, Y_2, \ldots, Y_N$. Dado que la salida es una estimación estadística, es imperativo calcular no solo la media, sino también el error estándar de la estimación para determinar la precisión de la simulación.

$$\text{Error Estándar} = \frac{S_Y}{\sqrt{N}} \tag{6}$$

Donde S_Y es la desviación estándar de la muestra de salidas.

4.6. Implementación de la Arquitectura en Python

A continuación, se presenta un código que abstrae estos elementos en una estructura de clase, demostrando buenas prácticas de programación científica.

```python
import numpy as np

class SimuladorMonteCarlo:
    def __init__(self, n_simulaciones):
        self.n = n_simulaciones
        self.resultados = []

    def generar_entradas(self):
        """
        Elemento 1, 2 y 3: Definicion, RNG y Muestreo.
        Retorna un diccionario con los valores aleatorios para una iteracion.
        """
        # Ejemplo: inputs para un sistema de inventario
        return {
            "demanda": np.random.poisson(lam=50),  # Distribucion discreta
            "costo_unitario": np.random.uniform(10, 12) # Distribucion continua
        }

    def modelo_sistema(self, inputs):
        """
        Elemento 4: Modelo Computacional (Logica determinista).
        """
        demanda = inputs["demanda"]
        costo = inputs["costo_unitario"]
        precio_venta = 20 # Parametro fijo

        # Logica del negocio
        ingreso = demanda * precio_venta
        egreso = demanda * costo
        utilidad = ingreso - egreso
        return utilidad

    def ejecutar(self):
        """
        Ciclo principal de simulacion
        """
        resultados_temp = []
        for _ in range(self.n):
            # Paso A: Generar escenario
            escenario = self.generar_entradas()
            # Paso B: Evaluar modelo
            salida = self.modelo_sistema(escenario)
            resultados_temp.append(salida)

        self.resultados = np.array(resultados_temp)

    def reporte(self):
        """
        Elemento 5: Agregacion y Analisis.
        """
        media = np.mean(self.resultados)
        error_std = np.std(self.resultados) / np.sqrt(self.n)

        print(f"--- Reporte de Simulacion (N={self.n}) ---")
        print(f"Media Estimada: {media:.4f}")
        print(f"Error Estandar: {error_std:.4f}")
        print(f"Intervalo Confianza 95%: [{media - 1.96*error_std:.4f}, {media + 1.96*error_std:.4f}]")
```

```python
# Uso de la estructura
sim = SimuladorMonteCarlo(n_simulaciones=5000)
sim.ejecutar()
sim.reporte()
```

Listing 2: Arquitectura Orientada a Objetos para Monte Carlo

5. Ejemplo Aplicado: Análisis de Riesgo en Utilidades

En la ingeniería financiera y comercial, rara vez trabajamos con valores fijos. Los precios, la demanda y los costos fluctúan. A continuación, modelaremos la utilidad esperada de un nuevo producto sujeto a incertidumbre.

5.1. Definición del Problema

Una empresa desea lanzar un producto tecnológico. Basado en datos históricos y estudios de mercado, se han determinado los siguientes parámetros estocásticos:

- **Precio de Venta (P):** Fijo en \$50 por unidad.
- **Costo Variable Unitario (C_v):** Incierto. Sigue una distribución Uniforme entre \$20 y \$30.
- **Costos Fijos (C_f):** \$50,000.
- **Demanda Anual (D):** Incierta. Sigue una distribución Normal con media $\mu = 3000$ unidades y desviación estándar $\sigma = 500$.

La utilidad (U) se define como:

$$U = (P - C_v) \times D - C_f \tag{7}$$

El objetivo no es encontrar un solo valor de utilidad, sino entender la distribución de posibles utilidades y la probabilidad de obtener pérdidas (Riesgo).

5.2. Simulación Numérica

Realizaremos 10,000 iteraciones para estimar la utilidad esperada y la probabilidad de pérdida ($U < 0$).

```python
import numpy as np

def simulacion_utilidad(n_simulaciones):
    # Parametros fijos
    precio = 50
    costo_fijo = 50000

    # Generacion de variables aleatorias vectorizadas
    # Costo Variable ~ Uniforme(20, 30)
    costo_variable = np.random.uniform(20, 30, n_simulaciones)

    # Demanda ~ Normal(3000, 500)
    demanda = np.random.normal(3000, 500, n_simulaciones)

    # Calculo de la utilidad para cada escenario
```

```python
    # El calculo es vectorizado (elemento a elemento)
    utilidad = (precio - costo_variable) * demanda - costo_fijo

    # Analisis de resultados
    utilidad_esperada = np.mean(utilidad)
    desviacion_est = np.std(utilidad)
    prob_perdida = np.sum(utilidad < 0) / n_simulaciones

    return utilidad_esperada, desviacion_est, prob_perdida

# Ejecutar la simulacion
n = 10000
media, desv, p_perdida = simulacion_utilidad(n)

print(f"Resultados de {n} simulaciones:")
print(f"Utilidad Esperada: ${media:,.2f}")
print(f"Desviacion Estandar: ${desv:,.2f}")
print(f"Probabilidad de Perdida: {p_perdida:.2%}")
```

Listing 3: Simulación de Utilidad Financiera con NumPy

5.3. Interpretación

A diferencia de un análisis determinista que usaría solo los promedios (Demanda = 3000, Costo = 25), la simulación nos revela la volatilidad. Si obtenemos una probabilidad de pérdida del 5 %, la gerencia puede decidir si ese riesgo es aceptable o si se deben tomar medidas para reducir los costos fijos o asegurar un precio de venta más alto.

6. Conclusión

La Simulación de Monte Carlo permite transformar la incertidumbre en los inputs en una distribución de probabilidad de los outputs. Es una herramienta poderosa para la toma de decisiones bajo riesgo. Sin embargo, su precisión depende de la calidad de los datos de entrada y de la correcta elección de las distribuciones de probabilidad que modelan las variables estocásticas.